

Завдання Блоку 2: Spring Boot and REST API

Контекст

Використовуйте предметну область, обрану під час виконання [Завдання 1](#).

Сутність 1 (основна для нас) - відноситься як many-to-one до Сутності 2.

В цьому завданні ви створюватиме новий проект, тому деякі атрибути чи їх типи можуть бути змінені, порівняно з тим як ви робили в першому завданні.

Завдання

Розробити Spring Boot сервіс, який зберігає в БД (Postgre) і надає доступ через REST API до даних Сутностей 1 і 2. Сервіс має реалізовувати наступні endpoints (замініть entity1 і entity2 на назви відповідних сутностей):

Endpoint	Що робить
POST /api/entity1 {...}	Створює новий запис Сутності 1. Валідувати обов'язкові поля, їх формат і т.і.
GET /api/entity1/{id}	Повертає детальні дані запису Сутності 1. Включаючи дані Сутності 2, на яку він посилається, наприклад "entity2": {"id": 2, "name": "MyValue"}
PUT /api/entity1/{id} {...}	Оновлює дані одного запису Сутності 1 по ID. Не забуваємо про валідацію.
DELETE /api/entity1/{id}	Видаляє запис Сутності 1 по ID
POST /api/entity1/_list { "entity2Id": 2, ..., "page": 1, "size": 20 }	Ендпоінт реалізує пагінацію списку елементів Сутності 1. Запит повертає структуру даних з елементами для запитаної сторінки, а також загальну кількість сторінок. Наприклад: { "list": [...], "totalPages": 5 } Записи в масиві матимуть скорочений набір полів Сутності 1 (бо не всі поля потрібні в режимі списку).

	В запиті опціонально можна вказати поля (2-3 поля), по яким можна фільтрувати записи (entity2 та ін.). Фільтрація має виконуватися на рівні запиту до БД, з урахуванням індексів на відповідних полях, щоб не погіршувати продуктивність при пагінації.
POST /api/entity1/_report <pre>{ "entity2Id": 2, ... }</pre>	Працює схожим чином, як і _list, але генерує і пропонує завантажити файл-репорт з усіма записами, які відповідають критеріям фільтрації (не лише однієї сторінки). Формат файлу: Excel або CSV (на ваш вибір). Використовуйте правильні заголовки відповіді, щоб браузер пропонував зберегти файл з правильним ім'ям файлу.
POST /api/entity1/upload	Приймає JSON-файл, для яких розроблювався парсер в Завданні 1 (формат можна адаптувати). Зберігає в БД всі валідні записи з цього файлу. У відповідь генерує JSON, в якому зазначає кількість успішно імпортованих записів, а також неуспішно (наприклад, не знайшли відповідний запис Сутності 2).
GET /api/entity2	Повертає список всіх наявних в БД записів Сутності 2
POST /api/entity2 <pre>{...}</pre>	Додає запис Сутності 2 Мати контроль унікальності найменувань Сутності 2 (не дозволяти створювати дублікати)
PUT /api/entity2/{id} <pre>{...}</pre>	Змінює запис Сутності 2 по ID. Враховуючи контроль унікальності найменувань Сутності 2
DELETE /api/entity2/{id}	Видаляє запис Сутності 2 по ID

Покрити інтеграційними тестами кожен endpoint.

Створити liquibase скрипт, який при запуску додатку створює необхідну схему БД, а також наповнення для Сутності 2.

Прикласти в репозиторії JSON-файл для імпорту даних (має бути узгодженим з liquibase скриптом наповнення).